Edge cases (a request that fails validation) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `sqlite_persistence_practice.py` (Edge cases section): "bad input never touches the database" is easy to believe and rarely checked. This section proves it with a real row count, then looks at the one case automatic checking can't cover — a valid request that fails between two writes. **If not:** the Build Task's "bad request body → 422, no database write" test case, and its started → completed/failed status updates, would rest on a guess.

The solution below assume your `sqlite_persistence` server is running on `localhost:8000` (or use `TestClient`, as noted).

Solution

Story: two `curl` outputs checked by eye prove it once, today. A script with `assert` proves it every time you run it, and stops loudly the day it breaks. **If not:** a future change that accidentally writes before checking input would go unnoticed.

```
# practice/sqlite_persistence_practice.py — Edge cases section
import sqlite3
import requests

conn = sqlite3.connect("runs.db")

def row_count() -> int:
    return conn.execute("SELECT COUNT(*) FROM runs").fetchone()[0]

before = row_count()

url = "http://localhost:8000/chat"
response = requests.post(url, json={})
# why: assert stops the script with this message if the check fails
assert response.status_code == 422, f"expected 422, got {response.status_code}"

after = row_count()
assert before == after, f"row count changed: {before} -> {after}"
```

```
print("422 confirmed, no row written")
print(response.json())
```

Expected output (the second line is shown wrapped onto 2 lines just to fit the page — really one line of output):

```
422 confirmed, no row written
{'detail': [{'type': 'missing', 'loc': ['body', 'message'],
  'msg': 'Field required', 'input': {}}]}
```